PROJECT 1 OBJECTIVE

The goal of this project is to learn how to write Structural Verilog code by converting circuit diagrams into Verilog. It also involves using ModelSim to test digital circuits and check their output through waveforms. Another important part of the project is combining different modules, like a binary multiplier and a 7-segment display decoder, and running the design on an FPGA board using Quartus II. Finally, a report is written to explain the steps taken, show the code and test results, and discuss any challenges faced.

INTRODUCTION to PROJECT 1

In this project, we used Structural Verilog, ModelSim, an FPGA board, and Quartus II. Each tool played an important role in different phases of the project.

Structural Verilog:

1. Used to describe the circuit by connecting different components in a modular way.
2 Helped in creating a representation of the design.

ModelSim:

1. Used for simulating the Verilog code to verify the circuit's functionality.
2. Allowed us to analyze waveform outputs and debug any issues before implementation.

FPGA Board:

1. Helped us test the final design in hardware and observe the results
2. Allowed us to verify that our implementation worked as expected outside of simulation.

Quartus II:

1. Used for the Verilog code and generating the necessary files for FPGA programming.
2. Provided tools for compiling, analyzing, and loading design onto the FPGA board

Our Verilog CODES

Inside of **Project 1a** file:

1) **logic_unit.v :**

C:/altera/project1/project1a/logic_unit.v - Default

```verilog
module logic_unit(
    input A,
    input B,
    output wire AND_out,    // A.B
    output wire OR_out,     // A+B
    output wire NAND_out,   // (A.B)'
    output wire NOR_out,    // (A+B)'
    output wire XOR_out,    // A?B
    output wire XNOR_out,   // A?B
    output wire NOT_A,      // A'
    output wire NOT_B       // B'
);

//we did gate instantiations
and  gate1(AND_out, A, B);
or   gate2(OR_out, A, B);
nand gate3(NAND_out, A, B);
nor  gate4(NOR_out, A, B);
xor  gate5(XOR_out, A, B);
xnor gate6(XNOR_out, A, B);
not  gate7(NOT_A, A);
not  gate8(NOT_B, B);

endmodule
```

2) logic_unit_tb.v :

C:/altera/project1/project1a/logic_unit_tb.v - Default *

```
module logic_unit_tb;

// These are testbench signals
reg A, B;
wire AND_out, OR_out, NAND_out, NOR_out, XOR_out, XNOR_out, NOT_A, NOT_B;

// We instantiate the logic unit module
logic_unit uut (
    .A(A),
    .B(B),
    .AND_out(AND_out),
    .OR_out(OR_out),
    .NAND_out(NAND_out),
    .NOR_out(NOR_out),
    .XOR_out(XOR_out),
    .XNOR_out(XNOR_out),
    .NOT_A(NOT_A),
    .NOT_B(NOT_B)
);

// These are test sequence
initial begin
    $display("A B | AND OR NAND NOR XOR XNOR NOT_A NOT_B");
    $monitor("%b %b |  %b   %b   %b    %b    %b    %b     %b     %b",
             A, B, AND_out, OR_out, NAND_out, NOR_out, XOR_out, XNOR_out, NOT_A, NOT_B);

    // Test all possible input combinations
    A = 0; B = 0; #10;
    A = 0; B = 1; #10;
    A = 1; B = 0; #10;
    A = 1; B = 1; #10;

    // End simulation
    $finish;
end

endmodule
```

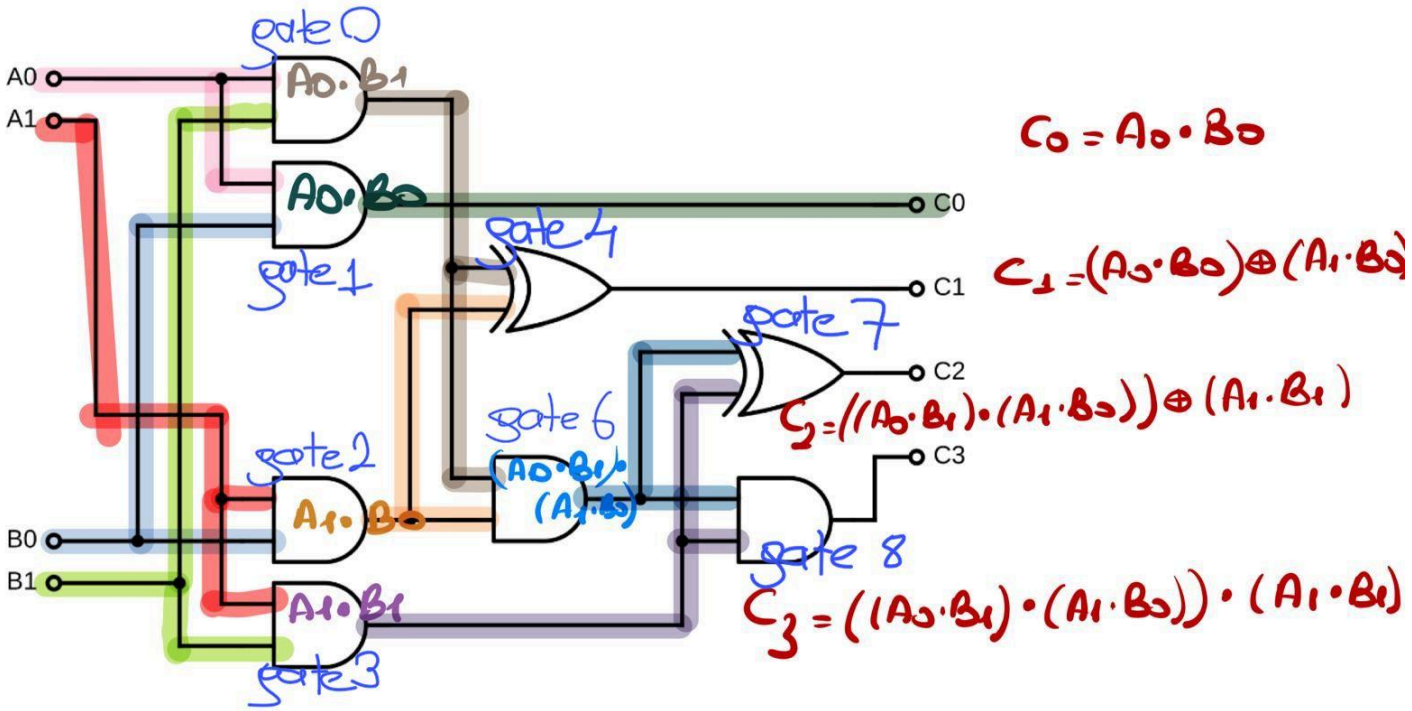
Inside of **Project 1b** file:

3) binary_multiplier.v:

C:/altera/project1/binary_multiplier.v - Default

```
module binary_multiplier(input [1:0] A, input [1:0] B,
output [3:0] C);
 wire p0, p1, p2, p3;
 and gate0(p0, A[0], B[1]);// these are the given gates
 and gate1(p1, A[0], B[0]);
 and gate2(p2, A[1], B[0]);
 and gate3(p3, A[1], B[1]);

 wire sum1,sum2, sum3,sum4;
 xor gate4(sum1,p0,p2);// and these are the gates we named
 and gate6(sum2,p0,p2);
 xor gate7(sum3,sum2,p3);
 and gate8(sum4,sum2,p3);

 assign C[0]=p1;  // we assigned outputs of final gates
 assign C[1]=sum1;
 assign C[2]=sum3;
 assign C[3]=sum4;
endmodule
```

we calculated this new gates (gate4, gate6, gate7, gate8) as below drawing,



4) binary_multiplier_tb.v:

C:/altera/project1/binary_multiplier_tb.v - Default *

```
module binary_multiplier_tb;

    // inputs and Outputs
    reg [1:0] A, B;
    wire [3:0] C;

    // instantiate the binary multiplier module
    binary_multiplier DUT (
        .A(A),
        .B(B),
        .C(C)
    );

    // test sequence
    initial begin
        $display("A  B  | C ");
        $monitor("%b  %b  | %b", A, B, C);

        // we tested all possible 2-bit combinations
        A = 2'b00; B = 2'b00; #10;
        A = 2'b00; B = 2'b01; #10;
        A = 2'b00; B = 2'b10; #10;
        A = 2'b00; B = 2'b11; #10;
        A = 2'b01; B = 2'b00; #10;
        A = 2'b01; B = 2'b01; #10;
        A = 2'b01; B = 2'b10; #10;
        A = 2'b01; B = 2'b11; #10;
        A = 2'b10; B = 2'b00; #10;
        A = 2'b10; B = 2'b01; #10;
        A = 2'b10; B = 2'b10; #10;
        A = 2'b10; B = 2'b11; #10;
        A = 2'b11; B = 2'b00; #10;
        A = 2'b11; B = 2'b01; #10;
        A = 2'b11; B = 2'b10; #10;
        A = 2'b11; B = 2'b11; #10;

        // end simulation
        $finish;
    end

endmodule
```

5) seg_decoder.v:

C:/altera/project1/seg_decoder.v - Default

```
module seg_decoder(input [3:0] num, output reg [6:0] seg);
    always @(*) begin
        case(num)
            4'd0: seg = 7'b1000000; // Display 0
            4'd1: seg = 7'b1111001; // Display 1
            4'd2: seg = 7'b0100100; // Display 2
            4'd3: seg = 7'b0110000; // Display 3
            4'd4: seg = 7'b0011001; // Display 4
            4'd5: seg = 7'b0010010; // Display 5
            4'd6: seg = 7'b0000010; // Display 6
            4'd7: seg = 7'b1111000; // Display 7
            4'd8: seg = 7'b0000000; // Display 8
            4'd9: seg = 7'b0010000; // Display 9
            default: seg = 7'b1111111; // Off
        endcase
    end
endmodule
```

6) project1b.v:

C:/altera/project1/project1b.v - Default

```
module project1b(
  input wire [1:0] A,
  input wire [1:0] B,
  output wire [6:0] seg);
    wire [3:0] C;

    // instantiate two-bit multiplier
    binary_multiplier mult (.A(A), .B(B), .C(C));

    // instantiate 7-segment decoder
    seg_decoder dec (.num(C), .seg(seg));
endmodule
```

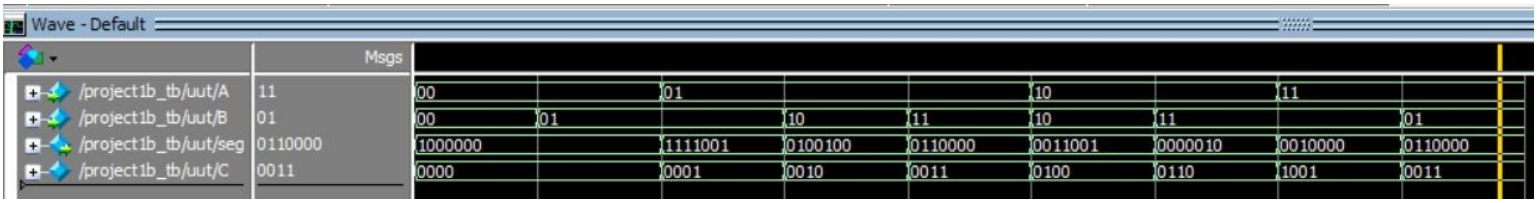
7) project1b_tb.v:

C:/altera/project1/project1b_tb.v - Default

```
module project1b_tb;
    reg [1:0] A, B;
    wire [6:0] seg;

    project1b uut (.A(A), .B(B), .seg(seg));

    initial begin
        $monitor("A=%b, B=%b, Segments=%b", A, B, seg);
        A = 2'b00; B = 2'b00; #10; // 0 * 0 = 0
        A = 2'b00; B = 2'b01; #10; // 0 * 1 = 0
        A = 2'b01; B = 2'b01; #10; // 1 * 1 = 1
        A = 2'b01; B = 2'b10; #10; // 1 * 2 = 2
        A = 2'b01; B = 2'b11; #10; // 1 * 3 = 3
        A = 2'b10; B = 2'b10; #10; // 2 * 2 = 4
        A = 2'b10; B = 2'b11; #10; // 2 * 3 = 6
        A = 2'b11; B = 2'b11; #10; // 3 * 3 = 9
        A = 2'b11; B = 2'b01; #10; // 3 * 1 = 3
        $finish;
    end
endmodule
```

Our Simulation Results

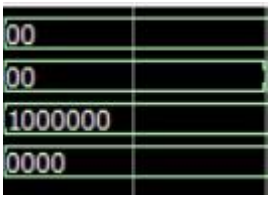
Our waveform result:



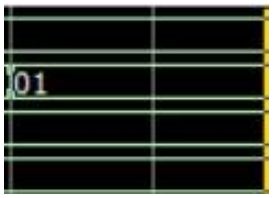
Our outputs in transcript:

```
Transcript
# Loading work.binary_multiplier
# Loading work.seg_decoder
add wave -position insertpoint sim:/projectlb_tb/uut/*
VSIM 5> run
# A=00, B=00, Segments=1000000
run
# A=00, B=01, Segments=1000000
run
# A=01, B=01, Segments=1111001
run
# A=01, B=10, Segments=0100100
run
# A=01, B=11, Segments=0110000
run
# A=10, B=10, Segments=0011001
run
# A=10, B=11, Segments=0000010
run
# A=11, B=11, Segments=0010000
VSIM 6> run
# A=11, B=01, Segments=0110000
# ** Note: $finish    : C:/altera/project1/projectlb_tb.v(18)
#    Time: 90 ps  Iteration: 0  Instance: /projectlb_tb
# 1
# Break in Module projectlb_tb at C:/altera/project1/projectlb_tb.v line 18
```

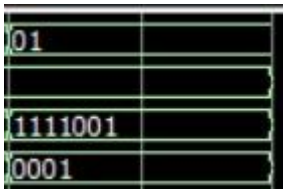
Explanations of what each waveform demonstrates:

1) 

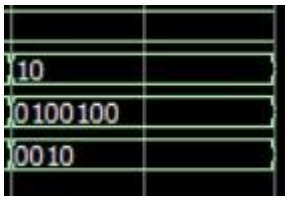
this waveform demonstrate
A=00, B=00, Segments=1000000
0 * 0 = 0

2) 

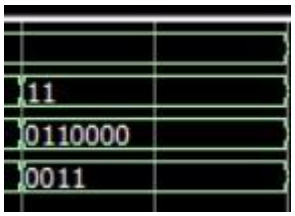
this waveform demonstrate
A=00, B=01, Segments=1000000
0 * 1 = 0

3) 

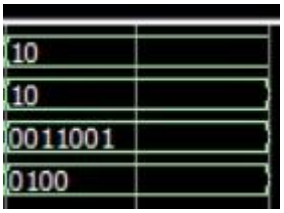
this waveform demonstrate
A=01, B=01, Segments=1111001
1 * 1 = 1

4) 

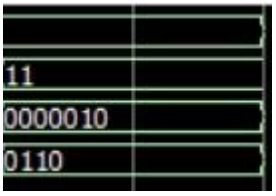
this waveform demonstrate
A=01, B=10, Segments=0100100
1 * 2 = 2

5) 

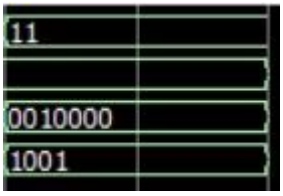
this waveform demonstrate
A=01, B=11, Segments=0110000
1 * 3= 3

6) 

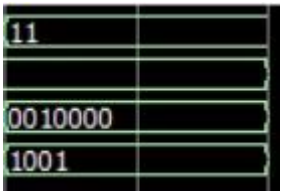
this waveform demonstrate
A=10, B=10, Segments=0011001
2 * 2 = 4

7) 

this waveform demonstrate
A=10, B=11, Segments=0000010
2 * 3= 6

8) 

this waveform demonstrate
A=11, B=11, Segments=0010000
3 * 3 = 9

9) 

this waveform demonstrate
A=11, B=01, Segments=0110000
3 * 1= 3

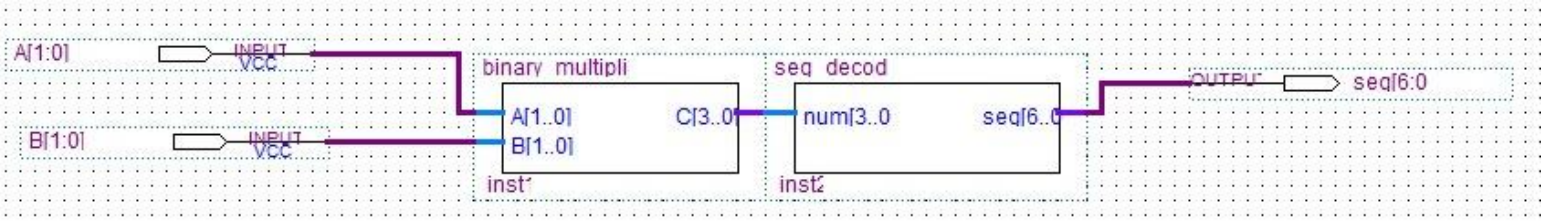
Our Quartus II Schematics

We added binary_multiplier.v, seg_decoder.v, and project1b.v in that respective order;

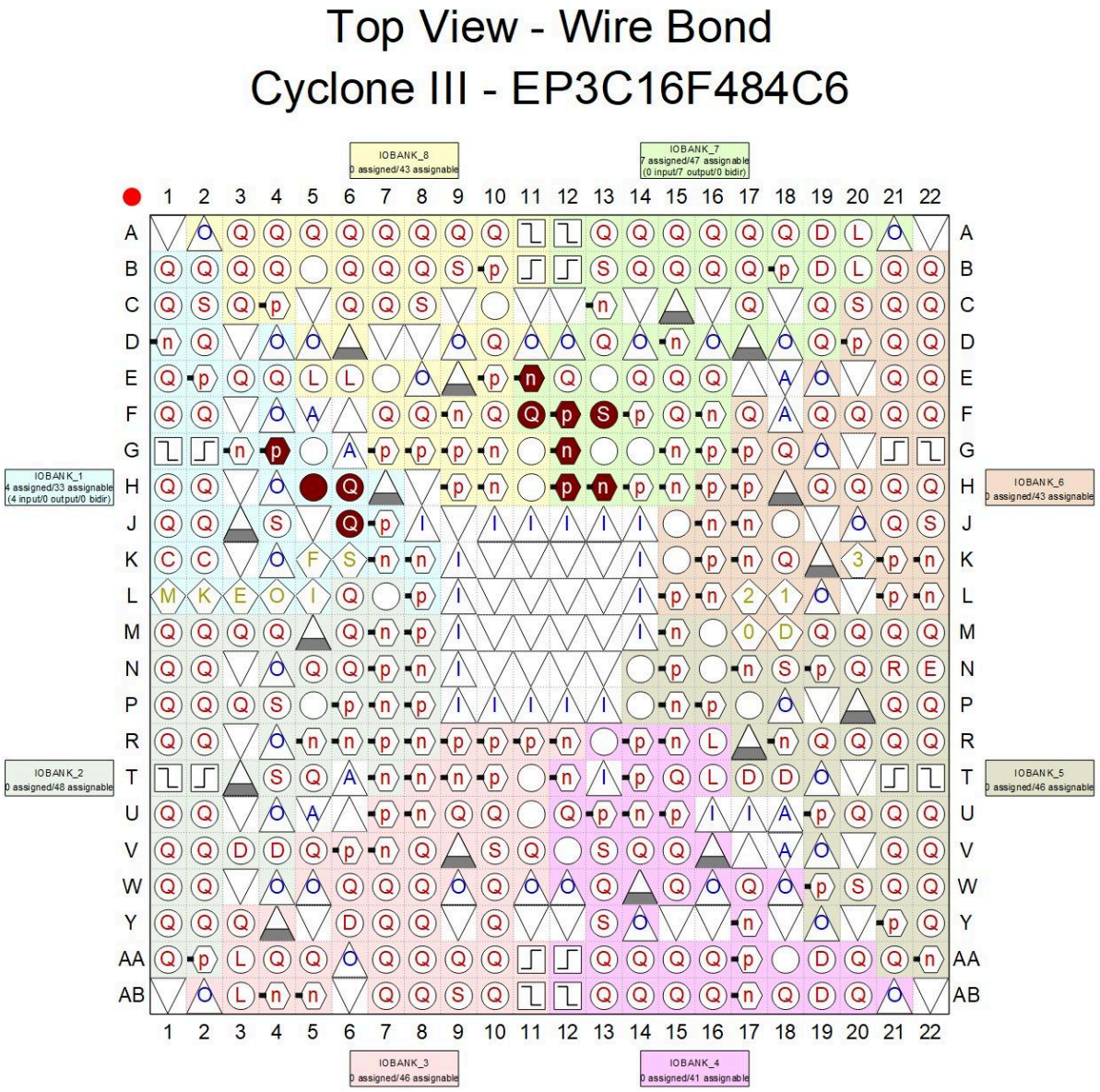
project1b_quartus.v

```
module binary_multiplier(input [1:0] A, input [1:0] B,
output [3:0] C);
 wire p0, p1, p2, p3;
 and gate0(p0, A[0], B[1]);
 and gate1(p1, A[0], B[0]);
 and gate2(p2, A[1], B[0]);
 and gate3(p3, A[1], B[1]);

 wire sum1,sum2, sum3,sum4;
 xor gate4(sum1,p0,p2);
 and gate6(sum2,p0,p2);//
 xor gate7(sum3,sum2,p3);
 and gate8(sum4,sum2,p3);

 assign C[0]=p1;
 assign C[1]=sum1;
 assign C[2]=sum3;
 assign C[3]=sum4;
endmodule

module seg_decoder(input [3:0] num, output reg [6:0] seg);
    always @(*) begin
        case(num)
            4'd0: seg = 7'b1000000; // Display 0
            4'd1: seg = 7'b1111001; // Display 1
            4'd2: seg = 7'b0100100; // Display 2
            4'd3: seg = 7'b0110000; // Display 3
            4'd4: seg = 7'b0011001; // Display 4
            4'd5: seg = 7'b0010010; // Display 5
            4'd6: seg = 7'b0000010; // Display 6
            4'd7: seg = 7'b1111000; // Display 7
            4'd8: seg = 7'b0000000; // Display 8
            4'd9: seg = 7'b0010000; // Display 9
            default: seg = 7'b1111111; // Off
        endcase
    end
endmodule

module project1b(input wire [1:0] A, input wire [1:0] B, output wire [6:0] seg);
    wire [3:0] C;

    // Instantiate two-bit multiplier
    binary_multiplier mult (.A(A), .B(B), .C(C));

    // Instantiate 7-segment decoder
    seg_decoder dec (.num(C), .seg(seg));
endmodule
```

This is the schematic we got;



This is our Pin Planner;



This is our all pins;

Named: * Edit:

Node Name	Direction	Location	I/O Bank	VREF Group	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair
A[1]	Input	PIN_H5	1	B1_N0	2.5 V (default)		8mA (default)		
A[0]	Input	PIN_J6	1	B1_N0	2.5 V (default)		8mA (default)		
B[1]	Input	PIN_G4	1	B1_N0	2.5 V (default)		8mA (default)		
B[0]	Input	PIN_H6	1	B1_N0	2.5 V (default)		8mA (default)		
seq[6]	Output	PIN_F13	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
seq[5]	Output	PIN_F12	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
seq[4]	Output	PIN_G12	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
seq[3]	Output	PIN_H13	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
seq[2]	Output	PIN_H12	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
seq[1]	Output	PIN_F11	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	
seq[0]	Output	PIN_E11	7	B7_N1	2.5 V (default)		8mA (default)	2 (default)	

All Pins

Summary

In this project we built and tested digital circuits using Structural Verilog, ModelSim, Quartus II, and FPGA board. We learned how simulation helps find errors before running the design on hardware. Our challenge was debugging errors in module connections but we fixed it by reviewing our code and using simulation. This project improved our understanding of digital circuits and the importance of testing and debugging.

How can we improve ourselves

In future projects, we can plan the design better, use testbenches more effectively, and keep track of errors to fix them faster. Learning more about FPGA features can also help improve our designs.

References

- DE0 Board User Manual, Terasic Technologies Inc.
- Digital Design with An Introduction to the Verilog HDL, 5th Edition, M. Morris Mano & Michael D. Ciletti, Pearson
- Quartus II Introduction Using Schematic Design, ALTERA.